

Observation Space

The observation is constructed fresh for each element as it is presented during an adaptation round. It is a flat vector of 8 scalars describing the element’s local error state, mesh structure, and global context. A 9th component (propagation likelihood) is reserved for the 2D extension.

Provenance key: **[DynAMO]** = adopted from Dzanic et al. (2024), **[Foucart]** = adopted from Foucart et al. (2023), **[Novel]** = novel to this architecture.

Component Summary

#	Component	Range	Dim	Provenance
0	α -normalized error	$[0, \infty)$	1	[DynAMO]
1	Left neighbor error	$[0, \infty)$	1	[DynAMO] / [Foucart]
2	Right neighbor error	$[0, \infty)$	1	[DynAMO] / [Foucart]
3	Refinement level	$[0, 1]$	1	[Novel]
4	Left neighbor level	$[0, 1]$	1	[Novel]
5	Right neighbor level	$[0, 1]$	1	[Novel]
6	Resource usage	$[0, 2]$	1	[Foucart]
7	Round progress	$[0, 1]$	1	[Novel]
8	Propagation likelihood	—	—	[DynAMO] (deferred to 2D)

The Error Indicator e_k

The raw error indicator for element k is the **average boundary jump magnitude** — the mean absolute solution discontinuity at the element’s two interfaces:

$$e_k = \frac{1}{2} \left(|u_h^{(k)}(x_L) - u_h^{(k-1)}(x_L)| + |u_h^{(k)}(x_R) - u_h^{(k+1)}(x_R)| \right) \quad (1)$$

where $u_h^{(k)}(x_L)$ is the solution evaluated at element k ’s left boundary from within element k , and $u_h^{(k-1)}(x_L)$ is the solution at the same point evaluated from within the left neighbor. The superscripts denote which element’s polynomial is being evaluated, not function powers.

For the DG method, the exact solution is continuous, so any nonzero interface jump is a measure of discretization error. An element that needs refinement will have large jumps at its boundaries; a well-resolved element will have jumps near zero. This indicator is in the same family as Foucart’s boundary non-conformity γ_K and DynAMO’s ZZ-type error estimator, simplified to raw jump magnitude for the 1D case.

α -Normalization (Components 0–2) **[DynAMO]**

The raw error e_k varies across many orders of magnitude depending on mesh resolution and waveform. To make the observation scale-invariant, we apply the α -normalized log-error transformation from DynAMO (Eq. 15):

$$o_k = \frac{-\log_{10}(e_k)}{\log_{10}(\alpha \cdot \|e\|_\infty)} \quad (2)$$

where:

- e_k is the element’s raw error indicator (Eq. ??),
- $\|e\|_\infty = \max_j e_j$ is the maximum error across all active elements,
- $\alpha \in (0, 1)$ is the error tolerance parameter (fixed at $\alpha_{\text{train}} = 0.1$ during training).

Why this works. Both numerator and denominator are in log-space relative to the global maximum error, so the observation is insensitive to absolute error magnitude and mesh resolution. The agent sees *relative* error position within the current mesh, not absolute values.

The decision boundary. Values cluster around 1.0 at the classification threshold. To see why, note that the refinement threshold is $e_{\text{max}} = \alpha \cdot \|e\|_\infty$. When $e_k = e_{\text{max}}$:

$$o_k = \frac{-\log_{10}(\alpha \cdot \|e\|_\infty)}{\log_{10}(\alpha \cdot \|e\|_\infty)} = 1.0$$

So $o_k > 1$ means the element’s error is above the refinement threshold (refinement candidate), and $o_k < 1$ means it’s below. The agent learns to act on this natural boundary.

Edge case. When $e_k = 0$ (or the denominator is zero), the value is clamped using a machine-epsilon floor: $e_k \leftarrow \max(e_k, \varepsilon)$ with $\varepsilon = 10^{-30}$.

Components 0–2: Error Information

Component 0: Element error [DynAMO]

$$o_0 = \frac{-\log_{10}(e_k)}{\log_{10}(\alpha \cdot \|e\|_\infty)}$$

The α -normalized error for the current element. This is the primary signal the agent uses to decide whether the element needs refinement or coarsening.

Component 1: Left neighbor error [DynAMO] / [Foucart]

$$o_1 = \frac{-\log_{10}(e_{k-1})}{\log_{10}(\alpha \cdot \|e\|_\infty)}$$

The same α -normalized error, evaluated for the left neighbor. In the periodic domain, the leftmost element’s left neighbor is the rightmost element. Adopted from Foucart’s Component A (neighbor boundary jumps provide spatial context), using DynAMO’s normalization.

Component 2: Right neighbor error [DynAMO] / [Foucart]

$$o_2 = \frac{-\log_{10}(e_{k+1})}{\log_{10}(\alpha \cdot \|e\|_\infty)}$$

Same as above for the right neighbor. The neighbor errors provide spatial context: the agent can assess whether the current element is a local error peak or part of a broader high-error region.

Components 3–5: Mesh Structure [Novel]

Neither DynAMO nor Foucart includes refinement level in the observation. DynAMO deliberately excludes it — their single-level architecture makes it unnecessary. We include it because the agent needs to predict cascade costs: refining an element whose neighbor is two levels coarser will trigger a cascade, and the level difference between neighbors is the signal for this.

Component 3: Element refinement level

$$o_3 = \frac{\ell_k}{\ell_{\max}}$$

where ℓ_k is the current refinement level of element k and $\ell_{\max}$ is `max_level`. Normalized to $[0, 1]$.

Component 4: Left neighbor refinement level

$$o_4 = \frac{\ell_{k-1}}{\ell_{\max}}$$

Component 5: Right neighbor refinement level

$$o_5 = \frac{\ell_{k+1}}{\ell_{\max}}$$

Together, components 3–5 let the agent compute level differences between the current element and its neighbors, which is the information needed to anticipate whether an action will trigger a 2:1 balance cascade.

Components 6–7: Global Context

Component 6: Resource usage [\[Foucart\]](#)

$$o_6 = \frac{|\mathcal{A}|}{B}$$

where $|\mathcal{A}|$ is the number of currently active elements and B is the element budget. Adapted from Foucart’s Component C, which communicates global resource state to local decisions. Without this, the agent would have no way to gauge how close it is to the budget constraint.

Values above 1.0 are possible after cascades — a refinement that triggers a cascade can push the mesh over budget. This is informative: the agent observes that the mesh is over-budget and can adjust subsequent decisions accordingly.

Component 7: Round progress [\[Novel\]](#)

$$o_7 = \frac{r}{R}$$

where r is the current round number (1-indexed) and R is the total number of rounds per remesh interval ($= \ell_{\max}$). This tells the agent where it is in the multi-round adaptation process. In early rounds, further refinement is still possible in later rounds; in the final round, the current decisions are the last opportunity before the solver advances.

Component 8 (Deferred): Propagation Likelihood [\[DynAMO\]](#)

DynAMO’s most distinctive observation feature: a non-dimensional quantity estimating the likelihood that a feature in a neighboring element will propagate to the current element within the remesh interval. For 1D constant-velocity advection, this quantity is constant for all element pairs and provides no discriminative information, so it is deferred until the system expands to 2D or variable-velocity problems where it becomes meaningful.

The α -Coupling Mechanism

The same parameter α appears in two places: the observation normalization (Eq. ??) and the reward classification thresholds ($e_{\max} = \alpha \cdot \|e\|_{\infty}$). During training, α is fixed at 0.1 and the agent learns a policy conditioned on what it sees through that lens.

At evaluation time, α can be freely varied without retraining. Changing α simultaneously shifts both what the agent “sees” and what it would be “judged by”:

- **Smaller** $\alpha \rightarrow$ the denominator in Eq. ?? decreases $\rightarrow$ normalized errors increase $\rightarrow$ more elements appear to be above the decision boundary $\rightarrow$ the agent refines more aggressively.
- **Larger** $\alpha \rightarrow$ the denominator increases $\rightarrow$ normalized errors decrease $\rightarrow$ fewer elements appear above the boundary $\rightarrow$ the agent refines less.

This gives a single knob to sweep the accuracy–cost tradeoff at deployment time: one trained policy produces a family of behaviors parameterized by α . Combined with sweeping the element budget (a second knob), this generates a 2D Pareto surface of accuracy versus cost.

Key Points

1. **Scale invariance through α -normalization.** The agent sees relative error position, not absolute magnitudes. The same policy works across different mesh resolutions and error scales.
2. **Decision boundary at $o = 1.0$.** The normalization is designed so that the refinement threshold maps exactly to $o = 1$, giving the agent a natural, learnable boundary.
3. **Refinement levels are novel and cascade-motivated.** DynAMO excludes level information; we include it because multi-level refinement with 2:1 balance makes cascade prediction essential.
4. **No raw solution values.** The original Masters thesis included solution values at LGL nodes, which caused a spurious correlation (the agent learned to refine where $u > 0$ rather than where gradients were steep). These are deliberately excluded.
5. **One trained policy, many behaviors.** The α -coupling lets a single trained agent produce a range of refinement strategies at deployment without retraining.